

This is a table of common issues the drone program might encounter over the runs. You are required to handle the **Mandatory** ones only. The optional ones might achieve a bonus.

The targets are which objects should handle the issue. For example, if we throw an exception, the target(s) catches it.

Gracefully handling issues is not that we don't throw an exception, is that we adjust our behavior to correct the issue or continue despite it. For example, we might separate a command into 2, log the occurrence as a warning and try to keep running.

Choosing the **N** value (number of tries, see below) is up to the implementation.

Scenario	Detector(s)	Action	Targets
All scenarios below, using a valid algorithm	Algorithm	A valid algorithm should avoid error that it can detect. Mandatory	The valid algorithm will not create the error.
A faulty algorithm sent a movement that would result in getting out of bounds	Drone controller	Gracefully ignore the illegal movement, do not pass it to the Movement component. Optional	Drone controller
A faulty algorithm returned a command with invalid values	Drone Controller	Gracefully try again, throw an exception after N tries. Optional	Drone Controller, SimulationRun
A faulty algorithm returns an empty movement and scan command ("NOOP")	Drone controller	Gracefully handle the empty request, try again. Throw after N failed attempts. Optional	Drone controller, SimulationRun
A faulty algorithm returns a movement that results with the drone colliding with a wall on the actual map.	Mock Movement driver (Which as a mock, holds the real map and can detect the collision).	Throw an exception. Mandatory	Drone Controller, SimulationRun
The LiDAR returns an empty vector	Drone controller	Gracefully try again, throw an exception after N tries. Optional	Drone Controller, SimulationRun
The movement driver returns "False" (for its own reasons)	Drone Controller	Gracefully handle the error, throw an exception after N tries. Optional	Drone Controller, SimulationRun

Scenario	Detector(s)	Action	Targets
The Algorithm returned a movement bigger than the max allowed	Drone controller	Handle gracefully and separate the movements into 2 or more commands, each as a separate step. Optional	Drone Controller
The drone returned an "Error" status on step	MissionControl	Log the error, continue. Optional	MissionControl
The Algorithm returned a movement that will take the drone out of the mission bounds	Drone Controller	Amend the movement so it will not take the drone out of the mission bounds. Optional	Drone Controller
The GPS returns out-of-bound coordinates	Drone Controller	Compare with internal coordinates and throw if also OOB, else ignore. Optional	Drone Controller, SimulationRun
A movement executed but GPS updated with impossible coordinates	Drone Controller	Gracefully try again to get GPS values, return Error on result after N tries. Optional	Drone Controller